Avaliação (Versão B) — Grafos, Algoritmos de Busca e NP-Completude

PCC104 — Projeto e Análise de Algoritmos

DECOM — ICEB — UFOP

*Instruções: Esta prova é individual e possui **5 questões**. É permitido consultar o código que você mesmo implementou nas listas de exercícios. Justifique todas as respostas — respostas sem justificativa não recebem pontuação*

1. Busca em grafos.

Considere a função de busca genérica cujo trecho principal é reproduzido abaixo. Cada vértice é armazenado como `[id, cor, pai, dist]`, com cores `'B'` (branco), `'C'` (cinza) e `'P'` (preto).

```
while F:
    if type == 'dfs':
        x = F.pop(-1)
    elif type == 'bfs':
        x = F.pop(0)
    if x[0] in O:
        return x
    for xi in G[x[0]]:
        if V[xi][1] == 'B':
            V[xi][1] = 'C'
            V[xi][2] = x[0]
            V[xi][3] = V[x[0]][3] + 1
            F.append(V[xi])
    V[x[0]][1] = 'P'
```

(a) Descreva o significado das três cores no ciclo de vida de um vértice durante a busca. Em particular, em que instante do laço um vértice passa de `'C'` (cinza) para `'P'` (preto), e o que significa um vértice estar em `'C'`?

(b) O teste `if V[xi][1] == 'B'` controla quais vizinhos entram em `F`. Explique por que esse teste é essencial: o que aconteceria com a execução (e com a corretude) se ele fosse removido e todo vizinho fosse sempre inserido em `F`?

2. Representação de grafos.

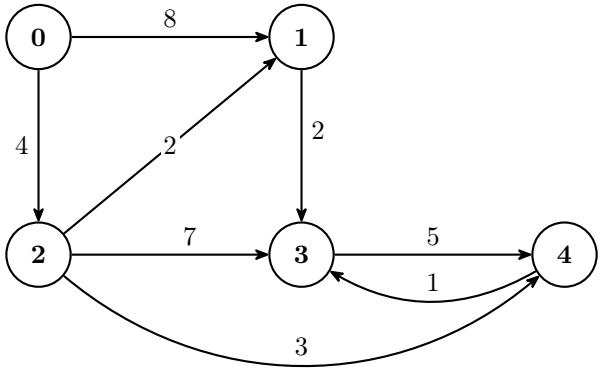
Considere um grafo com n vértices representado por uma matriz de adjacência M de dimensão $n \times n$.

(a) Descreva, em pseudocódigo ou em texto preciso, como implementar as operações `remover_aresta(u,v)` e `inserir_vertice(key)`. Indique a complexidade de cada uma em função de n.

(b) A mesma operação `inserir_vertice(key)`, se o grafo estivesse representado por lista de adjacência, teria complexidade diferente? Compare os dois casos justificando.

3. Caminhos mínimos (Dijkstra).

Considere o grafo direcionado ponderado abaixo, com origem no vértice 0.



(a) Execute o algoritmo de Dijkstra a partir de 0. Apresente uma tabela mostrando, a cada extração de vértice, a distância `d` e o predecessor `pai` de cada vértice. Indique a distância final até todos os vértices.

(b) Na sua implementação (`dijkstra.py`), a atualização só ocorre quando `v['d'] > u['d'] + w[(u,v)]`.

Explique o que essa condição representa e por que ela é essencial para a corretude do algoritmo.

4. NP-Completude.
Um estudante quer provar que Independent-Set é NP-difícil e afirma: *"Vou reduzir Independent-Set a Clique, que já sabemos ser NP-completo, e isso prova que Independent-Set é NP-difícil."* O argumento está correto? Aponte o erro (se houver) e indique qual é a direção correta da redução entre Clique e Independent-Set.

5. Vivendo com problemas NP-completos.
Um laboratório de computação precisa resolver, rotineiramente, instâncias de um problema NP-completo de escalonamento de tarefas em máquinas. Diante da impossibilidade de uma garantia polinomial de pior caso, a equipe adota abordagens diferentes conforme o contexto. Para cada situação abaixo, identifique qual das cinco estratégias da Seção 8 das notas está sendo empregada (algoritmo exponencial rápido na prática, caso especial tratável, algoritmo de aproximação, heurística/metaheurística, ou complexidade parametrizada) e justifique em uma frase.

(a) Para uma estimativa rápida de tempo total, a equipe aceita uma alocação comprovadamente no máximo 50% pior que a ótima, produzida por um procedimento garantidamente polinomial.
(b) Um cenário específico envolve tarefas cujas dependências formam sempre uma árvore; a equipe aplica um algoritmo próprio que resolve esse formato em tempo polinomial.
(c) Para o escalonamento noturno completo, roda-se um solver de *branch-and-bound* que termina em segundos na esmagadora maioria dos casos, ainda que o pior caso permaneça exponencial.
(d) Um subproblema envolve apenas até 4 máquinas críticas; usa-se um algoritmo cujo tempo é exponencial somente nesse número, mas linear no número total de tarefas.
(e) Sem qualquer garantia formal de qualidade, a equipe emprega um algoritmo genético que evolui as alocações e devolve a melhor solução encontrada dentro do orçamento de tempo.

Boa prova!